randproj1

March 31, 2026

1 Project 1 - Blnd Source Extraction using

2 The Problem

Let's consider a pair of signal sources, it could be two speakers playing different songs. If placed too close they will obviously interfere with eachother, generating a third signal. However, you'll notice that, assuming similar sound levels, you are able to pick out the songs out of the resulting signal, able to focus on one or the other.

The task is achieving just that, computationally.

3 Independent Component Analysis

Independent component analysis (ICA) is a computational method for separating a multivariate signal into additive subcomponents. This is done by assuming that at most one subcomponent is Gaussian and that the subcomponents are statistically independent from each other. ICA was invented by Jeanny Hérault and Christian Jutten in 1985. ICA is a special case of blind source separation.

4 The FastICA Algorithm

FastICA is an efficient and popular algorithm for independent component analysis invented by Aapo Hyvärinen at Helsinki University of Technology. Like most ICA algorithms, FastICA seeks an orthogonal rotation of prewhitened data, through a fixed-point iteration scheme, that maximizes a measure of non-Gaussianity of the rotated components. Non-gaussianity serves as a proxy for statistical independence, which is a very strong condition and requires infinite data to verify. FastICA can also be alternatively derived as an approximative Newton iteration.

4.1 Centering

4.2 Whitening:

Whitening transforms x to $4z$ with identity covariance: $z = Vx$ and $\mathbb{E}[zz^\top] = I$. This is done via the eigen-decomposition of C_x:

$$C_x = EDE^\top,$$

where E contains orthonormal eigenvectors and $D = \mathrm{diag}(\lambda_1, \dots, \lambda_n)$ the eigenvalues. The whitening matrix is

$$V = D^{-1/2} E^\top,$$

so the whitened data is $z = Vx$.

Example (4×4) — covariance and whitening:

$$C_x = \begin{pmatrix} 2.50 & 0.90 & 0.70 & 0.10 \\ 0.90 & 1.80 & 0.60 & 0.05 \\ 0.70 & 0.60 & 1.60 & 0.02 \\ 0.10 & 0.05 & 0.02 & 0.50 \end{pmatrix}.$$

Suppose its (numerical) eigen-decomposition yields

$$E = \begin{pmatrix} 0.77 & -0.53 & 0.34 & 0.06 \\ 0.57 & 0.65 & -0.47 & 0.05 \\ 0.29 & 0.54 & 0.78 & -0.08 \\ 0.02 & 0.04 & -0.02 & 0.99 \end{pmatrix}, \quad D = \mathrm{diag}(3.40,\ 1.25,\ 1.00,\ 0.75).$$

Then

$$D^{-1/2} = \mathrm{diag}(3.40^{-1/2},\ 1.25^{-1/2},\ 1.00^{-1/2},\ 0.75^{-1/2}) \approx \mathrm{diag}(0.54,\ 0.89,\ 1.00,\ 1.15),$$

and the whitening matrix is

$$V = D^{-1/2} E^\top.$$

Applied to the centered mixtures x, this produces whitened vectors $z = Vx$ with

$$\mathbb{E}[zz^\top] \approx I_4.$$

Whitening reduces the ICA search to an orthogonal matrix estimation.

In the actual implementation, the following algorithm was used:

X - input

X_w - Whitened output

C - covariance matrix of X

E - eigen vectors of C

D - eigen values of C

$X_w = D^{-1/2} E^T X$

4.3 Main Algorithm

For each unit vector w (a row of the final unmixing matrix), FastICA uses a nonlinear contrast function G with derivative $g = G'$ and second derivative g'. The symmetric (parallel) FastICA fixed-point update for a weight vector w on whitened data z is

$$w^{\text{new}} \leftarrow \mathbb{E}\{z\, g(w^\top z)\} - \mathbb{E}\{g'(w^\top z)\}\, w,$$

followed by normalization

$$w^{\text{new}} \leftarrow \frac{w^{\text{new}}}{\|w^{\text{new}}\|}$$

and decorrelation (for multiple components) via symmetric orthogonalization:

$$W \leftarrow (WW^\top)^{-1/2} W.$$

Common choices are: $g(u) = \tanh(u)$, $g(u) = u^3$, or $g(u) = u \exp(-u^2/2)$.

In this implementation $g(u) = \tanh(u)$ is the default and as such $g'(u) = 1 - \tanh^2(u)$

5 Importing and preprocessing data

4 sample signals are loaded form wav files. The signals are normalized ($E_s = 1$) to make it easier to compare them with the results.

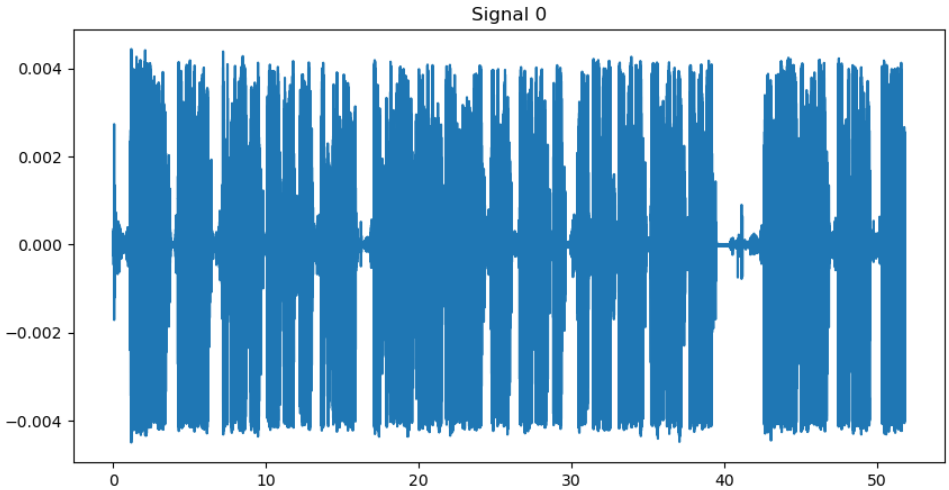
[1]:
```
# Helper functions
%matplotlib widget

from scipy.io import wavfile
import array
import scipy
import numpy as np
import matplotlib.pyplot as plt
from IPython.display import display, Markdown, Latex, Audio
from numpy import sqrt
from sklearn.decomposition import FastICA
from helpers import *

plt.style.use('default')
plt.interactive(True)
plt.rcParams['figure.figsize'] = [10, 5]

REFERENCE_MIX = "./data0proj.wav"

SIGNAL0_FILE = "./sample-01.wav"
SIGNAL1_FILE = "./sample-03.wav"
SIGNAL2_FILE = "./sample-02.wav"
```

```
SIGNAL3_FILE = "./sample-04.wav"

default_mix = [[1, 1], [0.5, 2]]
default_noise_level = 0.05
```

[2]:
```
# Reaading and generating data

signal0, times, signal0_samplerate = ReadWavFile(SIGNAL0_FILE)
signal1, times, signal1_samplerate = ReadWavFile(SIGNAL1_FILE)
signal2, times, signal2_samplerate = ReadWavFile(SIGNAL2_FILE)
signal3, times, signal3_samplerate = ReadWavFile(SIGNAL3_FILE)

assert signal0_samplerate == signal1_samplerate
assert signal2_samplerate == signal3_samplerate
assert signal1_samplerate == signal2_samplerate

signal0 = signal0 / np.linalg.norm(signal0)
signal1 = signal1 / np.linalg.norm(signal1)
signal2 = signal2 / np.linalg.norm(signal2)
signal3 = signal3 / np.linalg.norm(signal3)

samplerate = signal0_samplerate

display(Markdown("## Signal 0:"))
PlotAudio(signal0, samplerate, "Signal 0")

display(Markdown("## Signal 1:"))
PlotAudio(signal1, samplerate, "Signal 1")
```

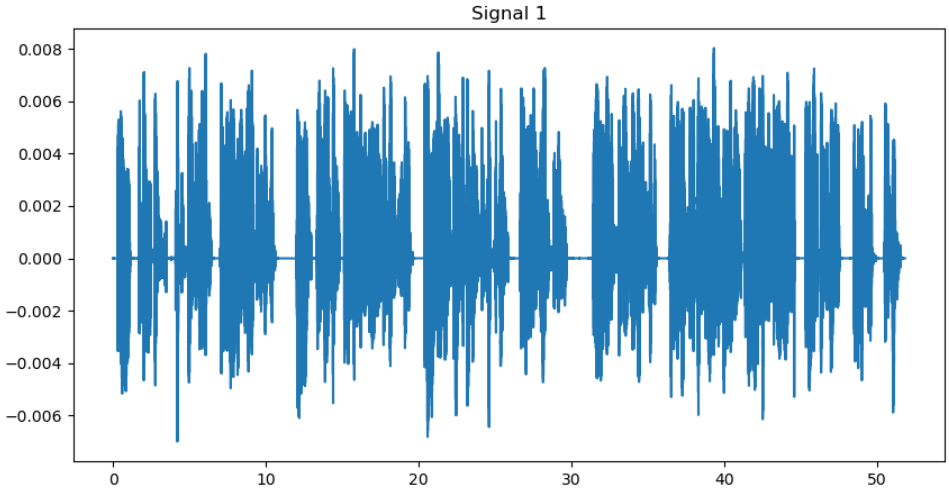
```
/home/user/Projects/randproj1/helpers.py:16: WavFileWarning: Chunk (non-data)
not understood, skipping it.
  samplerate, signal_raw = wavfile.read(path)
```

5.1 Signal 0:




```
<IPython.lib.display.Audio object>
```

5.2 Signal 1:

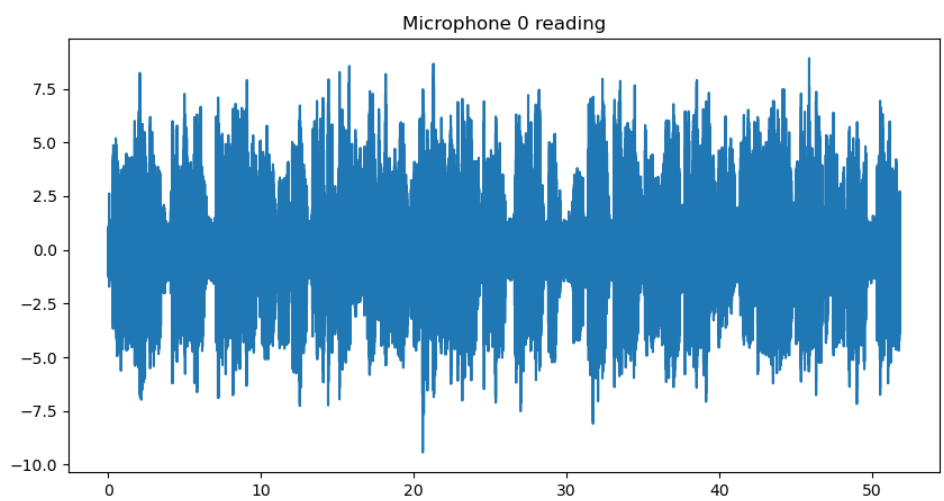



```
<IPython.lib.display.Audio object>
```

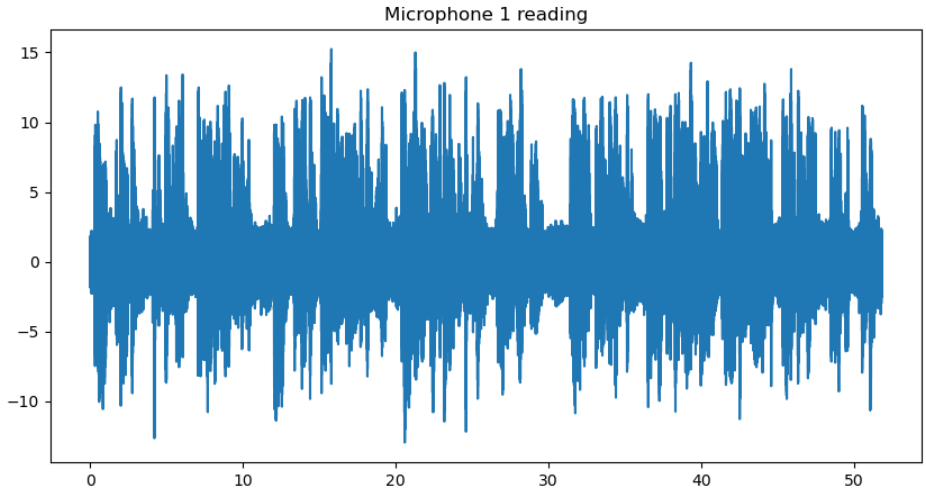
6 Generating microphone data

Signals are mixed to simulate a separate pair of microphones and whilte noise is injected in order to make the task more difficult

[3]:
```
# add noise
S = np.c_[AddNoise(signal0, default_noise_level), AddNoise(signal1,␣
↪default_noise_level)]

S /= S.std(axis=0)  # Standardize data

A = np.array(default_mix)  # Mixing matrix
X = np.dot(S, A.T)  # Generate observations

mic0 = X[:,0]
mic1 = X[:,1]

PlotAudio(mic0, samplerate, "Microphone 0 reading")
PlotAudio(mic1, samplerate, "Microphone 1 reading")
```




```
<IPython.lib.display.Audio object>
```



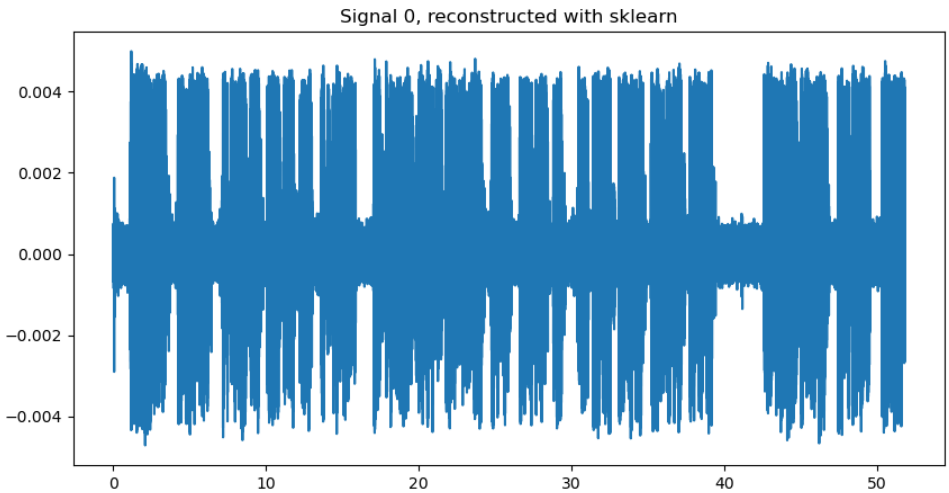

```
<IPython.lib.display.Audio object>
```

7 Baseline - the sklearn library:

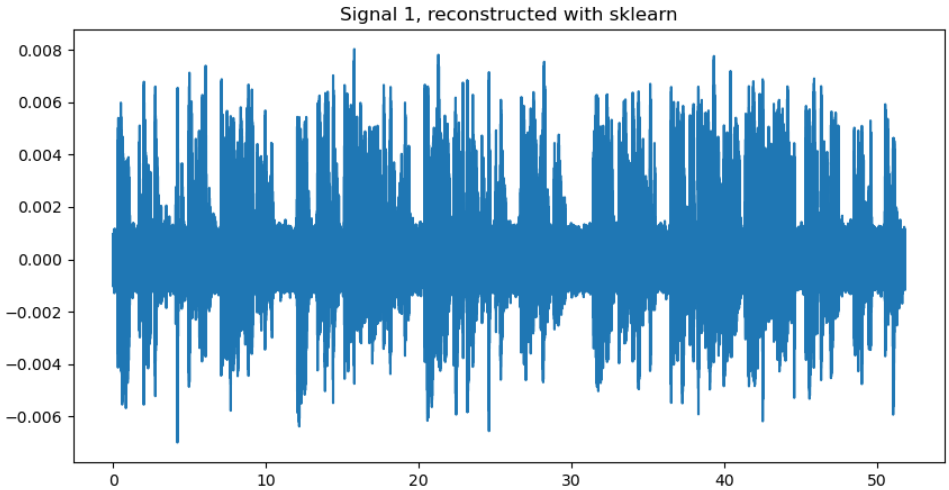
In order to generate a baseline for benchmarking the implementation, SKLearn is utilized to generate example outputs.

[4]:
```
# Reference test

ica = FastICA(n_components=2, whiten="arbitrary-variance")
S_ = ica.fit_transform(X)  # Reconstruct signals
A_ = ica.mixing_  # Get estimated mixing matrix

assigned = AssignDecomposed([signal0, signal1], [S_[:, 0] / np.linalg.norm(S_[:
 ↪, 0]), S_[:, 1] / np.linalg.norm(S_[:, 1])])
reconstructed0 = assigned[0]
reconstructed1 = assigned[1]
assigned = None

PlotAudio(reconstructed0, samplerate, "Signal 0, reconstructed with sklearn")

PlotAudio(reconstructed1, samplerate, "Signal 1, reconstructed with sklearn")

display(Markdown(f"Reference quality for mixing matrix {default_mix},␣
 ↪noise={default_noise_level}"))

qref0, _, _ = PlotDecompositionQuality(signal0, reconstructed0, "signal 0")
qref1, _, _ = PlotDecompositionQuality(signal1, reconstructed1, "signal 1")
```

```
# conserve RAM
reconstructed0 = None
reconstructed1 = None
```

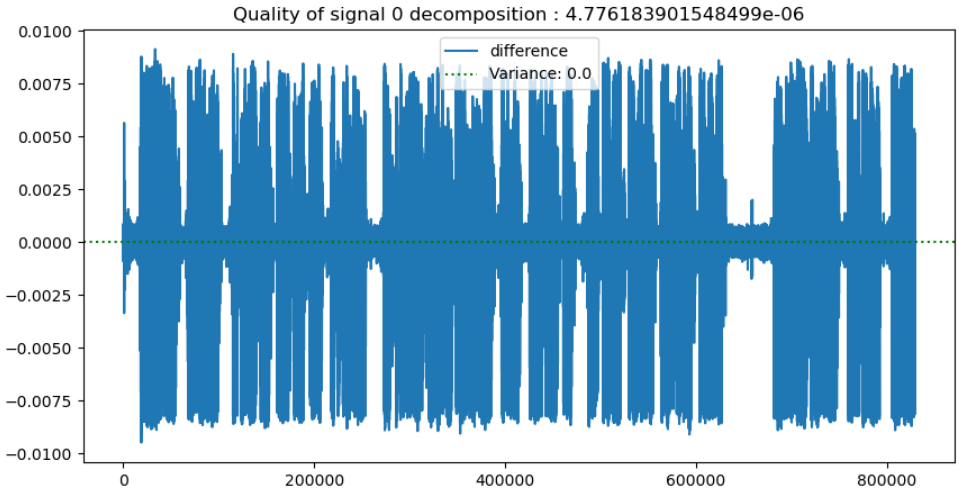


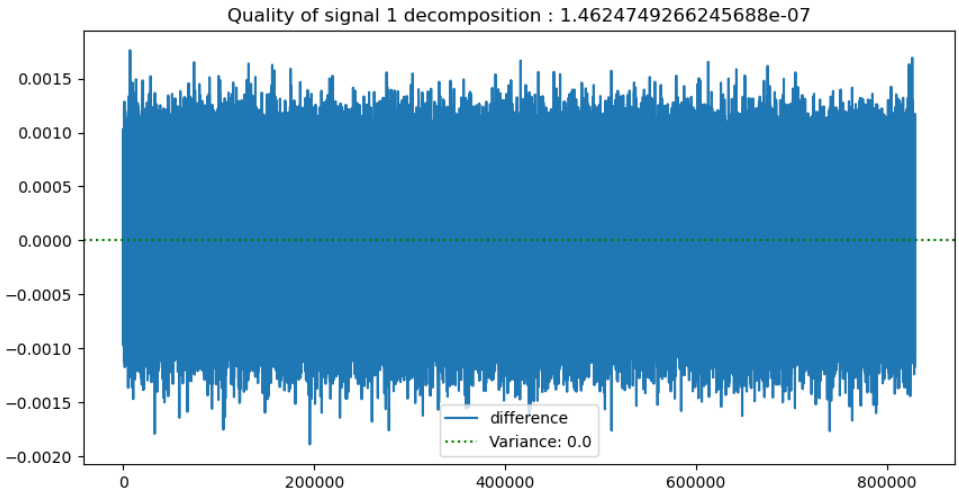

```
<IPython.lib.display.Audio object>
```




```
<IPython.lib.display.Audio object>
```

Reference quality for mixing matrix [[1, 1], [0.5, 2]], noise=0.05





8 Actually implementing the algorithm

The algorithm works as follows:

Input: C — number of desired components
Input: $\mathbf{X} \in \mathbb{R}^{N \times M}$ — prewhitened data (each column is an N-dimensional sample), $C \leq N$
Output: $\mathbf{W} \in \mathbb{R}^{N \times C}$ — unmixing matrix (columns project $\mathbf{X}$ onto independent components)
Output: $\mathbf{S} \in \mathbb{R}^{C \times M}$ — independent components, columns are M samples in C dimensions

$$
\begin{aligned}
&\textbf{for } p = 1, \dots, C: \\
&\quad \mathbf{w}_p \leftarrow \text{random vector in } \mathbb{R}^N \\
&\quad \textbf{while } \mathbf{w}_p \text{ changes:} \\
&\qquad \mathbf{w}_p \leftarrow \frac{1}{M}\,\mathbf{X}\,g(\mathbf{w}_p^T\mathbf{X})^T - \frac{1}{M}\,g'(\mathbf{w}_p^T\mathbf{X})\,\mathbf{1}_M\,\mathbf{w}_p \\
&\qquad \mathbf{w}_p \leftarrow \mathbf{w}_p - \sum_{j=1}^{p-1}(\mathbf{w}_p^T\mathbf{w}_j)\,\mathbf{w}_j \\
&\qquad \mathbf{w}_p \leftarrow \frac{\mathbf{w}_p}{\|\mathbf{w}_p\|} \\
&\quad \textbf{end while} \\
&\textbf{end for} \\
&\mathbf{W} \leftarrow \begin{bmatrix}\mathbf{w}_1 & \dots & \mathbf{w}_C\end{bmatrix}, \qquad \mathbf{S} \leftarrow \mathbf{W}^T\mathbf{X}
\end{aligned}
$$

Notes: - $\mathbf{X} \in \mathbb{R}^{N\times M}$ (N components, M samples).
- g is applied elementwise; g' denotes its derivative.
- $\mathbf{1}_M$ denotes an $M \times M$ matrix of ones (used here for sample averaging).

Of course $\mathbf{w}_p$ Will aways change, if only due to floating point imprecision. A difference is obtained using the following algorithm: $1 - |\mathbf{w}_p\mathbf{w}_old^T|$, the function terinates when this difference drops below a set threshold or when an interation limit is exceeded.

Additionally should the difference equal NaN as can happen in extreme circumstances, the matrix is randomly regenerated and the function continues as normal.

[5]:
```
from numpy import nan
import sys
import time
epsilon = sys.float_info.epsilon
def g(data):
    return np.tanh(data)
def g_prime(data):
    return 1 - np.tanh(data)**2
def g_both(data):
    g_ret = np.tanh(data)
    g_prime_ret = 1 - g_ret**2
    return g_ret, g_prime_ret

def FastICA(readings: np.ndarray, maxiter=1000, tolerance = 1e-7, showtime =␣
 ↪True, returnmatrices = False, showdifs = True, g_and_prime = g_both, whiten␣
 ↪= True, center = True, showiterations = True):
    start = time.time()
```

```
n = readings.shape[0]
nsamples = readings.shape[1]
signals = []

# center
for i in range(0, n):
    signal : np.ndarray = readings[i, :]
    if center:
        signal -= np.mean(signal) # zero the mean
    signals.append(signal)
S = np.array(signals)

Sw = S
if whiten:
    # whiten the input data
    cov = np.cov(S)
    evals, evecs = np.linalg.eigh(cov)
    inv = np.diag(1.0 / np.sqrt(evals + epsilon))
    #whitened input
    Sw = inv @ evecs.T @ S
iters = 0

Wplist = []

onescol = np.array([[1]] * nsamples)
Wmats = []
for p in range(0, n):
    Wp = np.random.uniform(size=(n, 1))
    i = 0
    while i < maxiter:
        if returnmatrices:
            Wplist.append(Wp)
        WX = Wp.T @ Sw
        gW, gW_prime = g_and_prime(WX)
        L = (1/nsamples)*(Sw @ gW.T)
        R = (1/nsamples) * (gW_prime @ onescol) @ Wp.T
        Wnew = L - R.T
        for w in Wmats:
            w = np.array([w]).T
            Wnew -= (Wnew.T @ w) * w
        Wnew = Wnew / np.linalg.norm(Wnew)
        dif = 1 - abs((Wp.T @ Wnew).item())
        if np.isnan(dif):
            Wp = np.random.uniform(size=(n, 1))
            print("Invalid iteration, got nan dif!")
        else:
            Wp = Wnew
```

```
            if showiterations:
                print(f"it: {i} dif: {dif}")
            if dif < tolerance:
                break
            i += 1
        if i == maxiter:
            print("Maximum number of iterations exceeded!")
        Wmats.append(np.array([Wp[i][0] for i in range(0, n)]))
        iters += i
    Wmats = np.array(Wmats)
    res = Wmats @ Sw
    # normalize the samples
    for i in range(0, res.shape[0]):
        res[i] = res[i, :] / np.linalg.norm(res[i, :])

    end = time.time()
    elapsed = end-start
    if showtime:
        display(Markdown(f"### Decomposed {n} inputs into {n} outputs in␣
↪{iters} iterations, elapsing {elapsed}s"))

    if returnmatrices:
        return res, iters, Wplist
    return res, iters
```

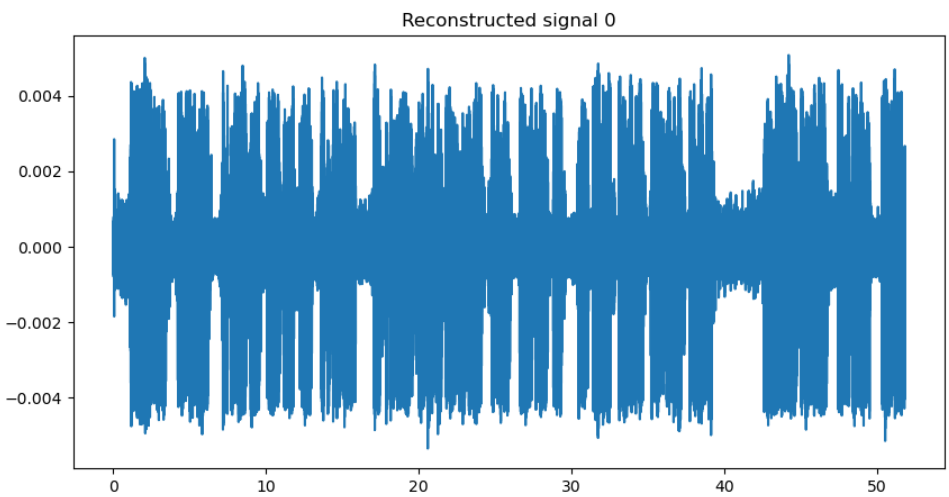
9 Test I: two signals

[6]:
```
res, n_iters = FastICA(np.array([mic0, mic1]))

assigned = AssignDecomposed([signal0, signal1], [res[0, :], res[1, :]])
r0 = assigned[0]
r1 = assigned[1]
assigned = None

PlotAudio(r0, samplerate, "Reconstructed signal 0")

PlotAudio(r1, samplerate, "Reconstructed signal 1")

display(Markdown(f"Quality for mixing matrix {default_mix},␣
↪noise={default_noise_level}"))


q0, _, _ = PlotDecompositionQuality(signal0, r0, "signal 0")
display(Markdown(f"Reference quality for signal 0 = {qref0}"))
qdif0 = q0 - qref0
```

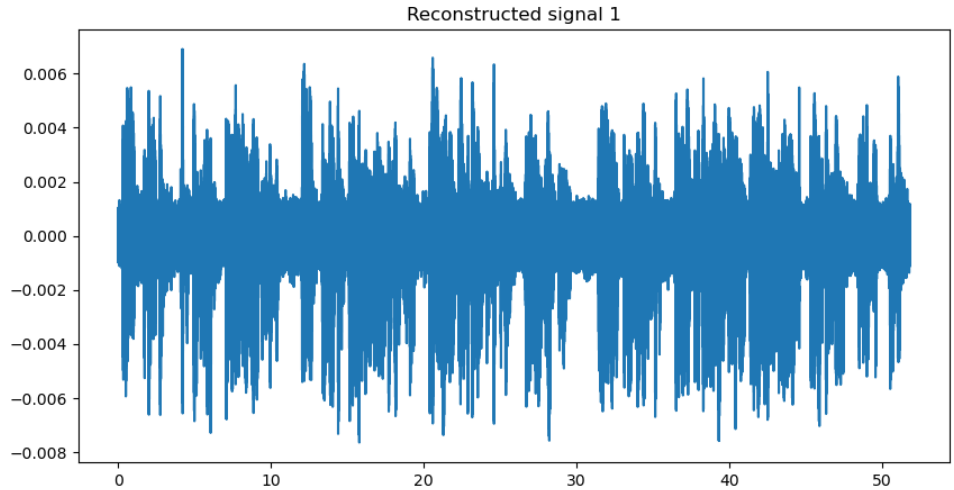
```
display(Markdown(f"Achieved quality for signal 0 = {q0} ({"+" if qdif0 > 0 else␣
↪""}{qdif0})"))

q1, _, _ = PlotDecompositionQuality(signal1, r1, "signal 1")
display(Markdown(f"Reference quality for signal 1 = {qref1}"))
qdif1 = q1 - qref1
display(Markdown(f"Achieved quality for signal 1 = {q1} ({"+" if qdif1 > 0 else␣
↪""}{qdif1})"))

r0 = None
r1 = None
```

```
it: 0 dif: -0.10511464926479541
it: 0 dif: 0.7772523113123923
it: 1 dif: 0.0
```

9.0.1 Decomposed 2 inputs into 2 outputs in 1 iterations, elapsing 0.276843786239624s

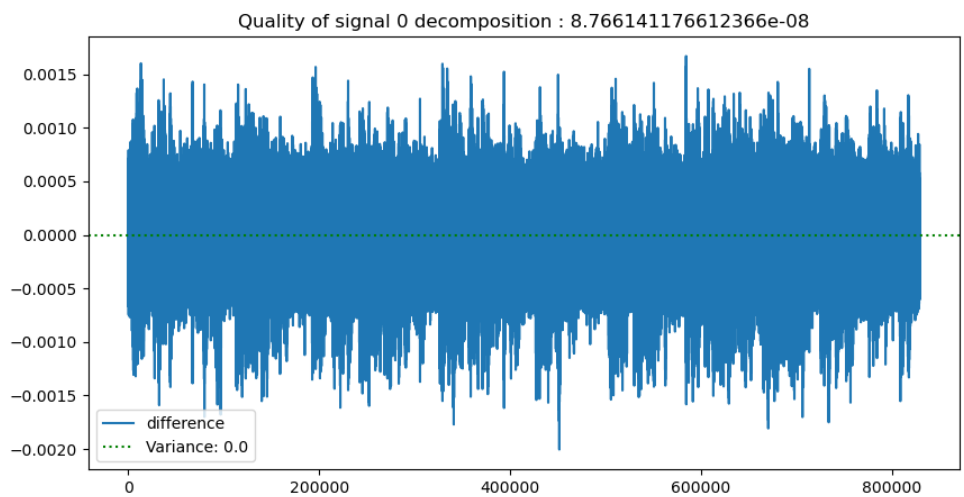



```
<IPython.lib.display.Audio object>
```



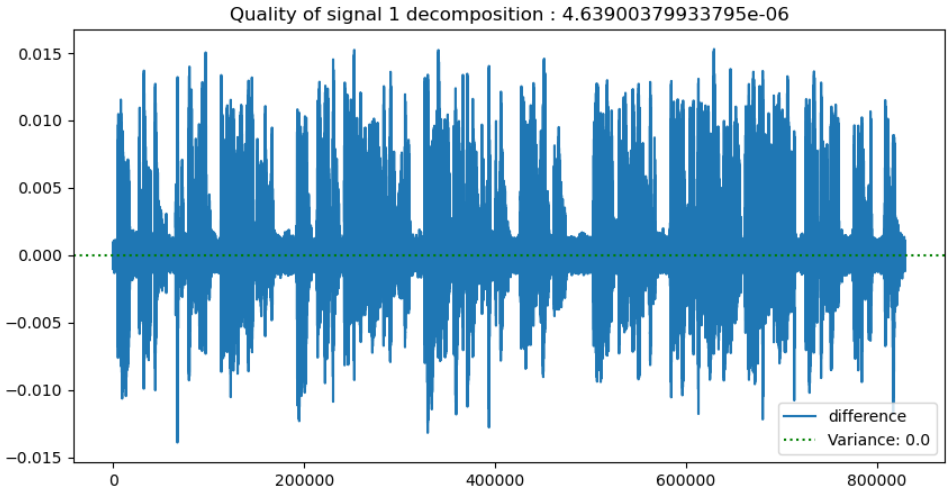

```
<IPython.lib.display.Audio object>
```

Quality for mixing matrix [[1, 1], [0.5, 2]], noise=0.05



Reference quality for signal 0 = 4.776183901548499e-06

Achieved quality for signal 0 = 8.766141176612366e-08 (-4.688522489782375e-06)



Reference quality for signal 1 = 1.4624749266245688e-07

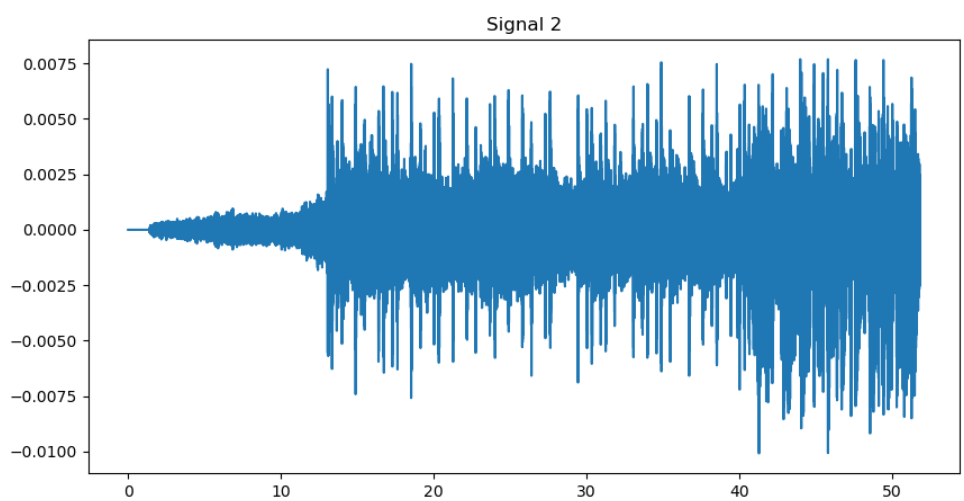
Achieved quality for signal 1 = 4.63900379933795e-06 (+4.492756306675493e-06)

10 Test II - 3 signals

[7]:
```
display(Markdown("## Signal 2:"))
PlotAudio(signal2, samplerate, "Signal 2")


default_mix = [[1, 1, 0.5], [0.5, 2, 1], [0.25, 1.5, 1.1]]
default_noise_level = 0.025

# add noise
S = np.c_[AddNoise(signal0, default_noise_level), AddNoise(signal1,␣
↪default_noise_level), AddNoise(signal2, default_noise_level)]

S /= S.std(axis=0)  # Standardize data

A = np.array(default_mix)  # Mixing matrix
X = np.dot(S, A.T)  # Generate observations

mic0 = X[:,0]
mic1 = X[:,1]
mic2 = X[:,2]


display(Markdown("Mic 0:"))
```

```
display(Audio(data=mic0, rate=samplerate))

display(Markdown("Mic 1:"))
display(Audio(data=mic1, rate=samplerate))

display(Markdown("Mic 2:"))
display(Audio(data=mic2, rate=samplerate))

res, n_iters = FastICA(np.array([mic0, mic1, mic2]), tolerance=1e-8)

assigned = AssignDecomposed([signal0, signal1, signal2], [res[0, :], res[1, :],␣
↪res[2, :]])
r0 = assigned[0]
r1 = assigned[1]
r2 = assigned[2]
assigned = None

PlotAudio(r0, samplerate, "Reconstructed signal 0")
PlotAudio(r1, samplerate, "Reconstructed signal 1")
PlotAudio(r2, samplerate, "Reconstructed signal 2")
```

10.1 Signal 2:




```
<IPython.lib.display.Audio object>
```

Mic 0:

```
<IPython.lib.display.Audio object>
```

Mic 1:

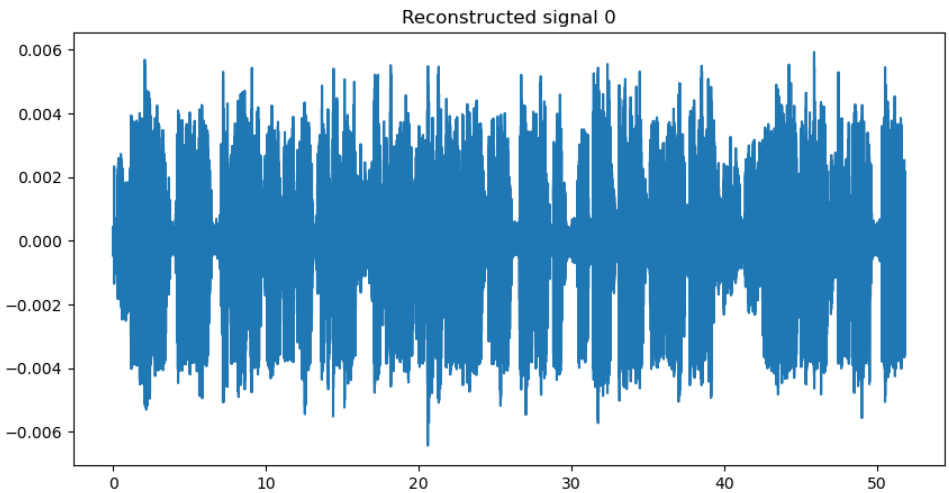
```
<IPython.lib.display.Audio object>
```

Mic 2:

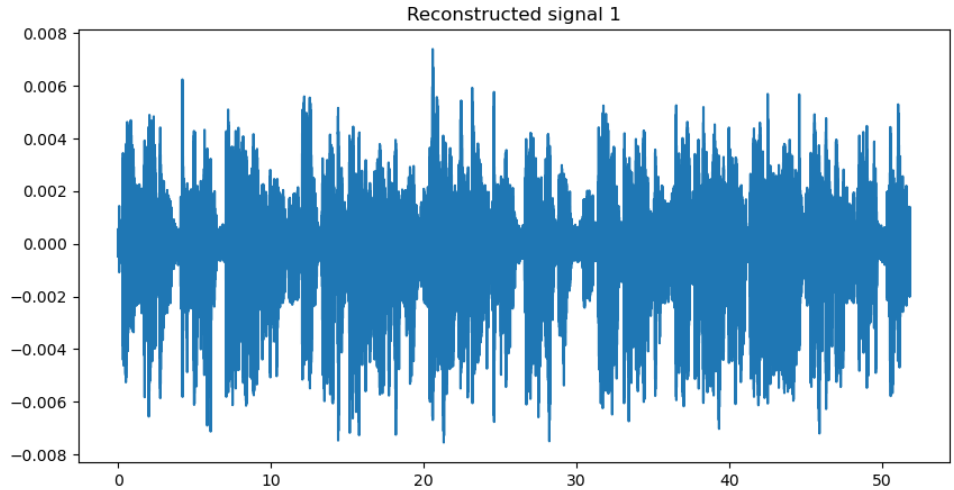
```
<IPython.lib.display.Audio object>
```

```
it: 0 dif: -0.08767919031959259
it: 0 dif: 0.34441168655700094
it: 1 dif: 0.0007170959005267097
it: 2 dif: 1.4977610329758306e-07
it: 3 dif: 4.679434617571587e-11
it: 0 dif: 0.675160289965984
it: 1 dif: 0.0
```

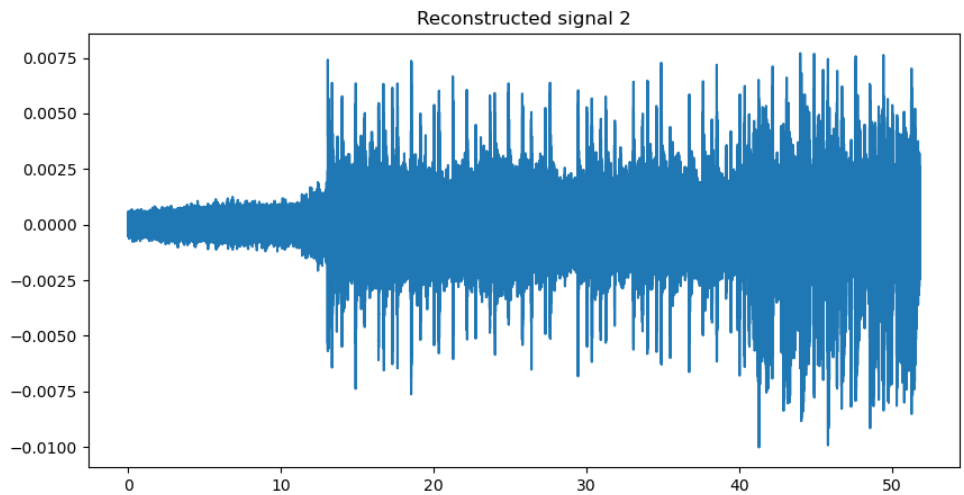
10.1.1 Decomposed 3 inputs into 3 outputs in 4 iterations, elapsing 0.6862099170684814s




```
<IPython.lib.display.Audio object>
```




```
<IPython.lib.display.Audio object>
```



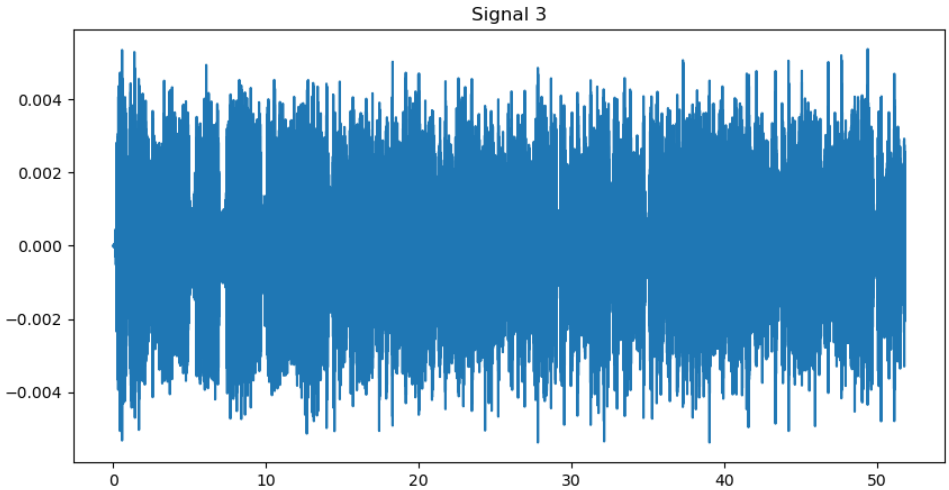

```
<IPython.lib.display.Audio object>
```

11 Test III - 4 signals

[8]:
```
display(Markdown("## Signal 3:"))
PlotAudio(signal3, samplerate, "Signal 3")
```

```
default_mix = [[0.1, 1, 1, 0.5], [3, 0.5, 2, 1], [1.1, 0.25, 1.5, 1.1], [0.9, 1.
↪8, 0.13, 0.3]]
default_noise_level = 0.0125

# add noise
S = np.c_[AddNoise(signal0, default_noise_level), AddNoise(signal1,␣
↪default_noise_level), AddNoise(signal2, default_noise_level),␣
↪AddNoise(signal3, default_noise_level)]

S /= S.std(axis=0)  # Standardize data

A = np.array(default_mix)  # Mixing matrix
X = np.dot(S, A.T)  # Generate observations

mic0 = X[:,0]
mic1 = X[:,1]
mic2 = X[:,2]
mic3 = X[:,3]

S = None


display(Markdown("Mic 0:"))
display(Audio(data=mic0, rate=samplerate))

display(Markdown("Mic 1:"))
display(Audio(data=mic1, rate=samplerate))

display(Markdown("Mic 2:"))
display(Audio(data=mic2, rate=samplerate))

res, n_iters = FastICA(np.array([mic0, mic1, mic2, mic3]), tolerance=1e-9)

assigned = AssignDecomposed([signal0, signal1, signal2, signal3], [res[0, :],␣
↪res[1, :], res[2, :], res[3, :]])
r0 = assigned[0]
r1 = assigned[1]
r2 = assigned[2]
r3 = assigned[3]
assigned = None
res = None

PlotAudio(r0, samplerate, "Reconstructed signal 0")
PlotAudio(r1, samplerate, "Reconstructed signal 1")
PlotAudio(r2, samplerate, "Reconstructed signal 2")
PlotAudio(r3, samplerate, "Reconstructed signal 2")
```

11.1 Signal 3:




```
<IPython.lib.display.Audio object>
```

Mic 0:

```
<IPython.lib.display.Audio object>
```

Mic 1:

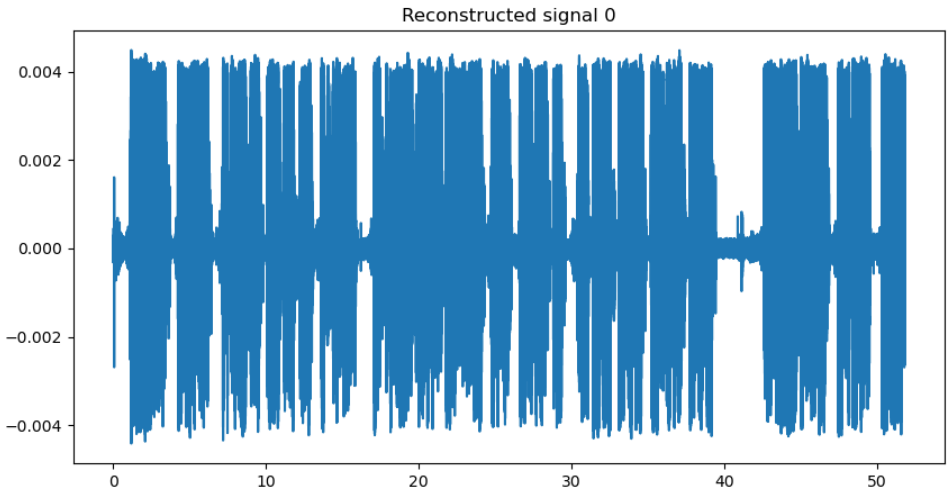
```
<IPython.lib.display.Audio object>
```

Mic 2:

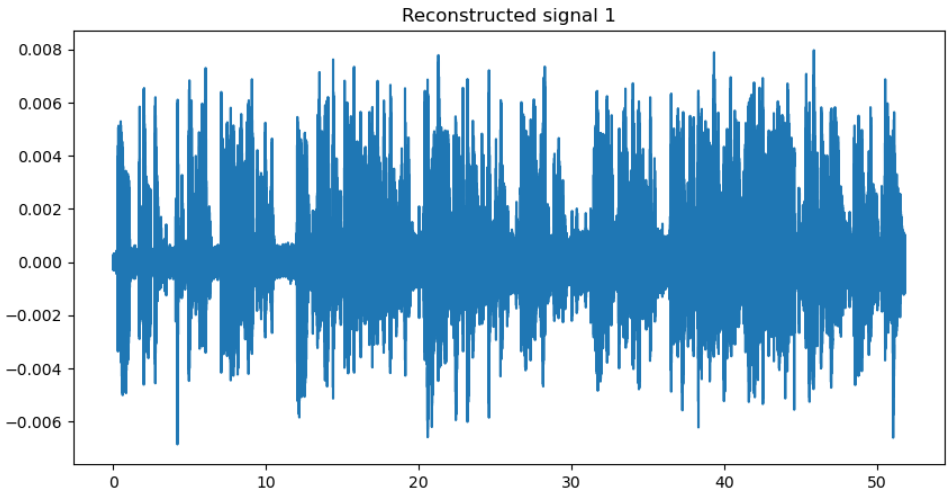
```
<IPython.lib.display.Audio object>
```

```
it: 0 dif: -0.01335833351982485
it: 0 dif: 0.6498742754714238
it: 1 dif: 0.047043305685051995
it: 2 dif: 0.00036515598338104915
it: 3 dif: 4.960687354227389e-07
it: 4 dif: 6.597695723087327e-10
it: 0 dif: -0.36899202563275635
it: 0 dif: 0.9857243568137134
it: 1 dif: 0.0
```

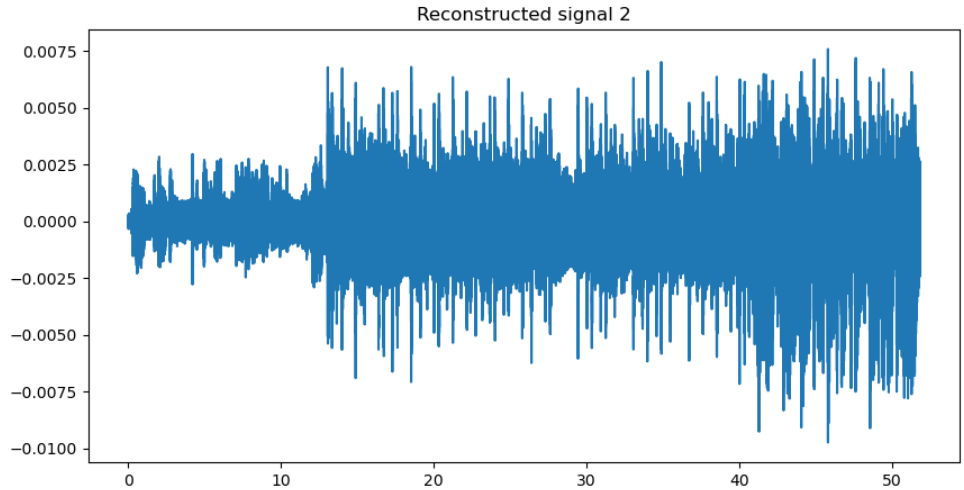
11.1.1 Decomposed 4 inputs into 4 outputs in 5 iterations, elapsing 0.5965206623077393s




```
<IPython.lib.display.Audio object>
```

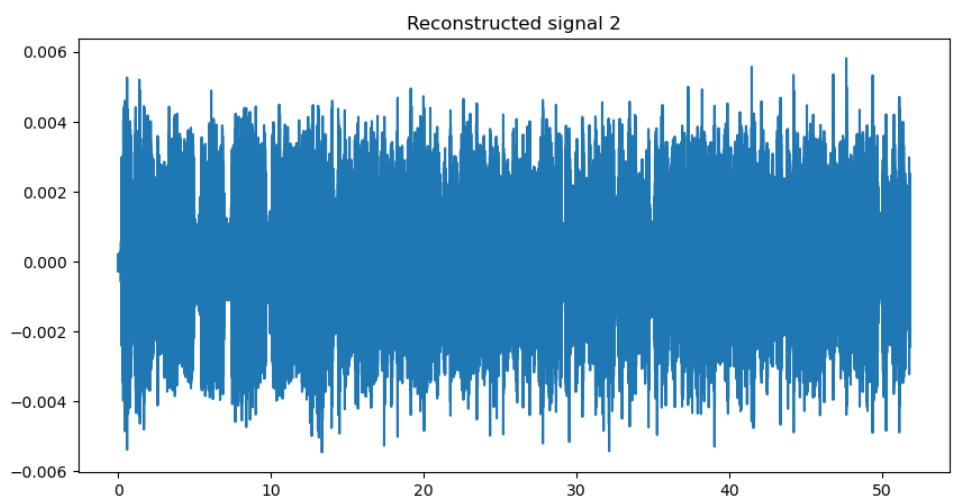



```
<IPython.lib.display.Audio object>
```




```
<IPython.lib.display.Audio object>
```

```
/home/user/Projects/randproj1/helpers.py:23: RuntimeWarning: More than 20
figures have been opened. Figures created through the pyplot interface
(`matplotlib.pyplot.figure`) are retained until explicitly closed and may
consume too much memory. (To control this warning, see the rcParam
`figure.max_open_warning`). Consider using `matplotlib.pyplot.close()`.
  fig = plt.figure()
```



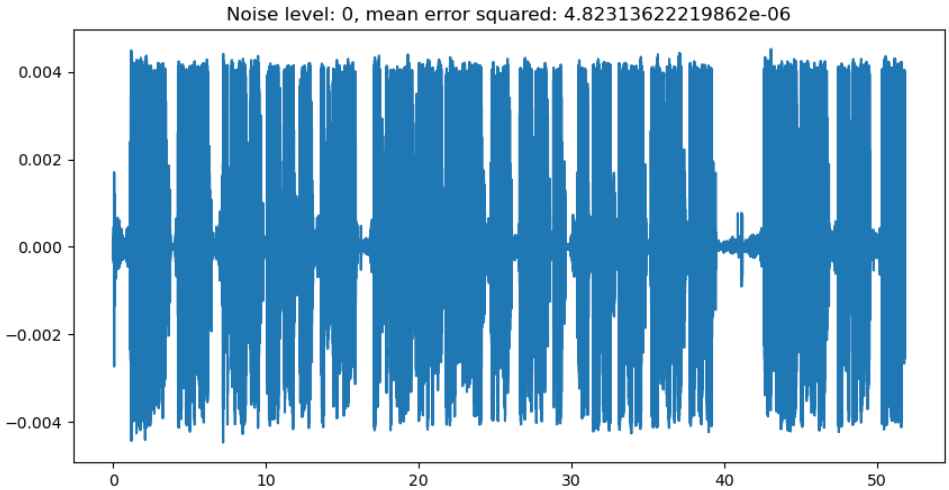

```
<IPython.lib.display.Audio object>
```

[8]:

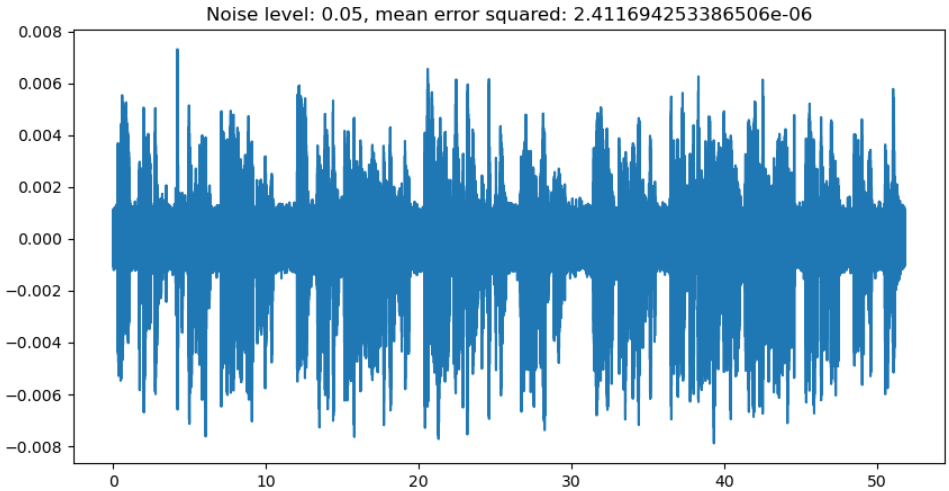
```
```

[9]:

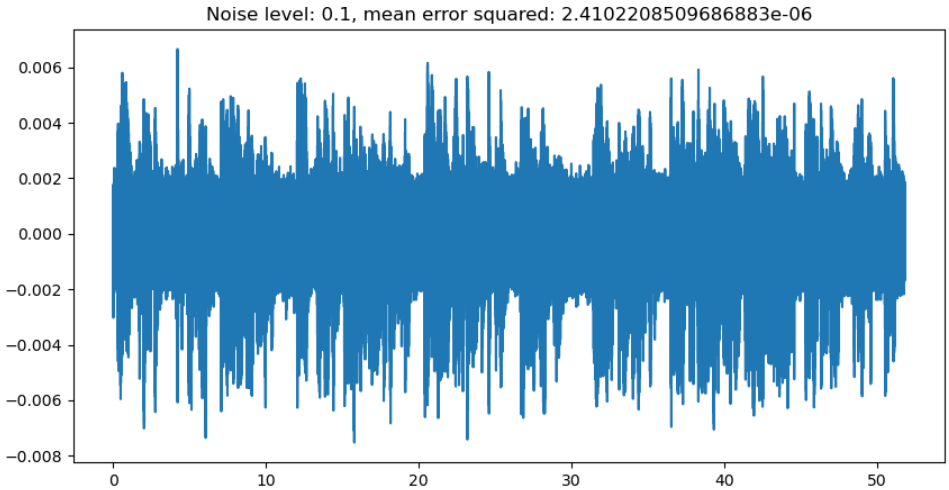
```
from matplotlib.widgets import Button, Slider

noiselevels = [0, 0.05, 0.1, 0.15, 0.25, 0.3, 0.4, 0.5, 0.8]
mix = [[0.5, 1.1], [1.5, 0.4]]
qualities = []

for nlevl in noiselevels:
    # add noise
    S = np.c_[AddNoise(signal0, nlevl), AddNoise(signal1, nlevl)]

    S /= S.std(axis=0)  # Standardize data

    A = np.array(mix)  # Mixing matrix
    X = np.dot(S, A.T)  # Generate observations

    mic0 = X[:,0]
    mic1 = X[:,1]

    res, n_iters = FastICA(np.array([mic0, mic1]), tolerance=1e-9,␣
 ↪showiterations=False, showtime=False)
    assigned = AssignDecomposed([signal0, signal1], [res[0, :], res[1, :]])
    r0 = assigned[0]
    r1 = assigned[1]
    assigned = None

    score, mean, diff = DecompositionQuality(signal0, r0)
    PlotAudio(r0, samplerate, f"Noise level: {nlevl}, mean error squared:␣
 ↪{score}")
    qualities.append(score)

# plt.figure()
# plt.title("Mean error for noise levels")
# plt.plot(noiselevels, qualities)
# plt.show()
```



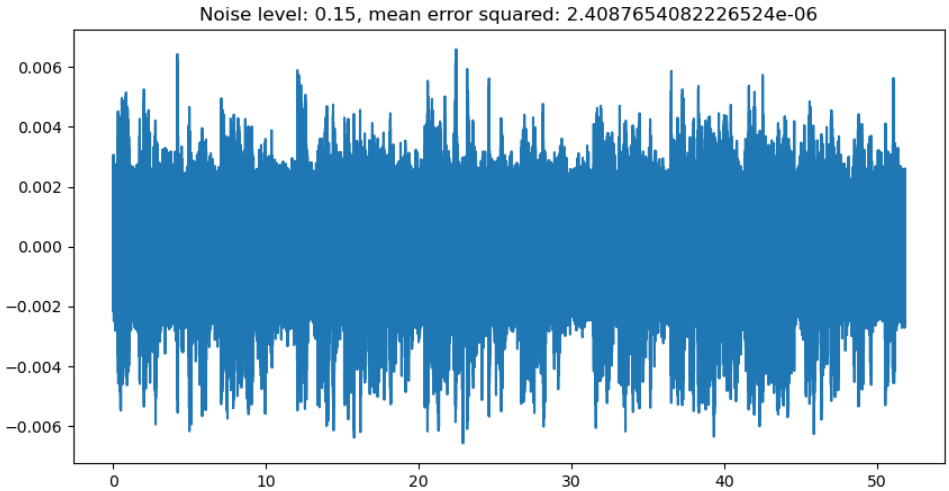

```
<IPython.lib.display.Audio object>
```



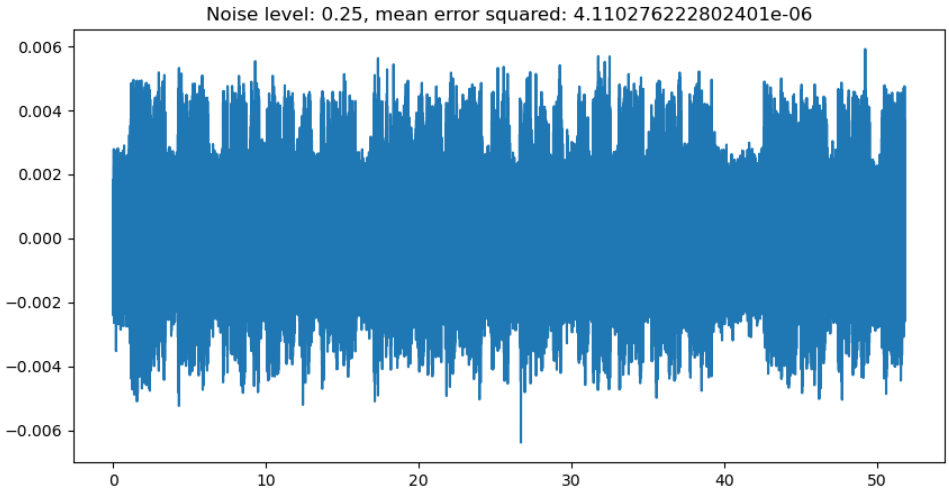

```
<IPython.lib.display.Audio object>
```




```
<IPython.lib.display.Audio object>
```

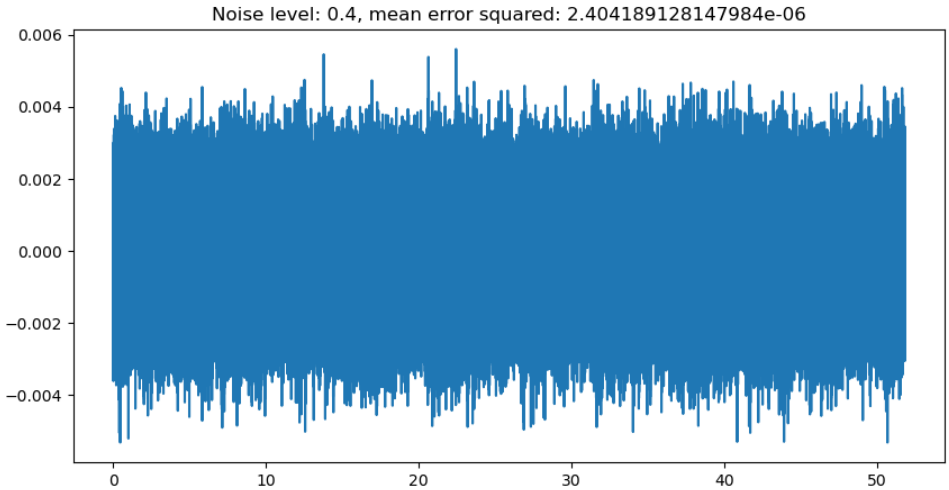



```
<IPython.lib.display.Audio object>
```

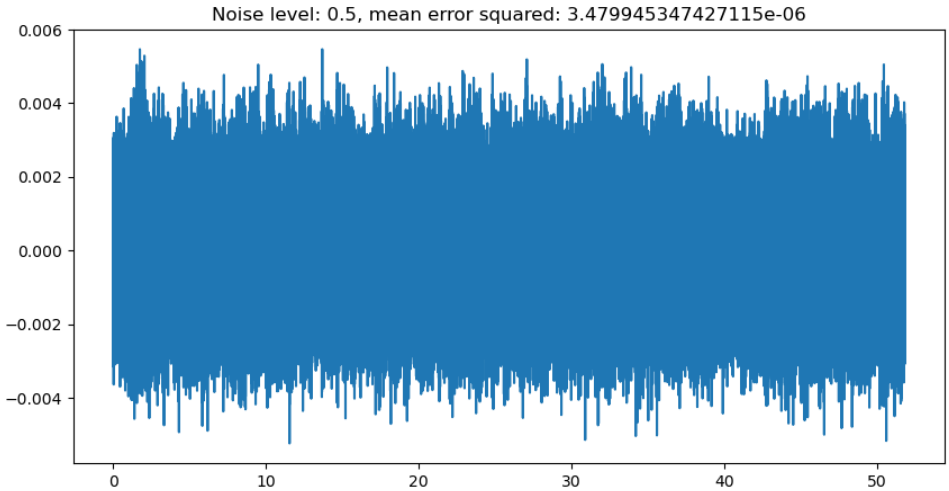



```
<IPython.lib.display.Audio object>
```

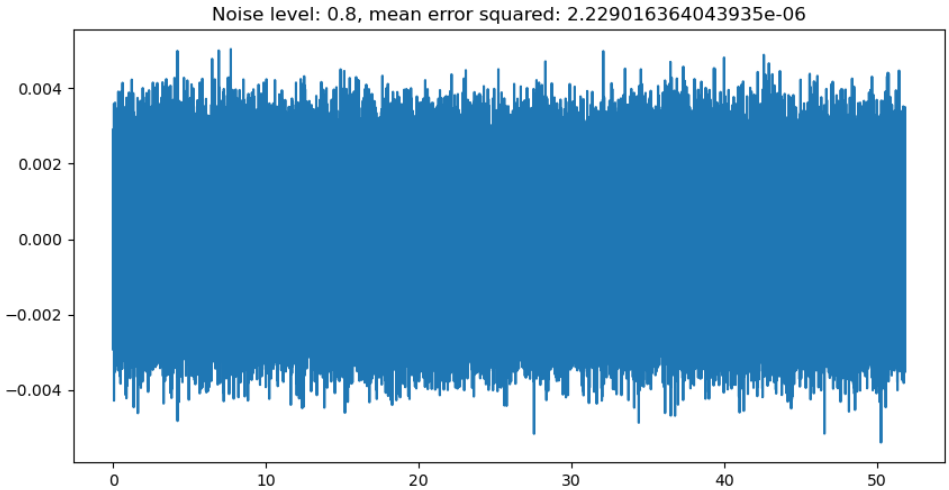
```
<IPython.lib.display.Audio object>
```




```
<IPython.lib.display.Audio object>
```




```
<IPython.lib.display.Audio object>
```




```
<IPython.lib.display.Audio object>
```

The algorithm is only really effective for noise levels below 0.15 (white noise, 0.15 is multiplied by the mean). Though quite unpleasant to listen to, the algorithm still manages to isolate the signals even up to the noise level of 0.8. An additional white noise removal step could be utilized to achieve much better results.

12 Conclussion

This report demonstrates a working FastIPA implementation. Further research could include using different functions for $g(x)$ as well as further comparison to the reference algorithm. A better algorithm for assigning output signals to inputs, which is needed to better understand the quality of the outputs, would also be necessary.